

Africa Credit Hub — Complete Bug Fix Guide for Replit

Document Version: 1.0

Date: April 2, 2026

Platform Stack: React / Next.js · i18n · PostgreSQL · Tailwind CSS

Total Bugs: 18 (4 Critical · 4 High · 6 Medium · 4 Low)

Table of Contents

#	Bug	Priority	Section
1	Broken Access Control	 Critical	Jump
2	XSS Vulnerability in Name Field	 Critical	Jump
3	Portfolio Excel Export Fails	 Critical	Jump
4	Credit Report Search Non-Functional	 Critical	Jump
5	Sidebar Navigation Not Translated	 High	Jump
6	Information Leakage on /profile and /settings	 High	Jump
7	Broken Terms/Privacy Links	 High	Jump
8	Incomplete Language Translations (AR/SW)	 High	Jump
9	Header Name Not Updated Real-Time	 Medium	Jump
10	CSV/Excel Export Column Mismatch	 Medium	Jump
11	Chain Integrity Verify No Feedback	 Medium	Jump
12	Raw Translation Keys Displayed	 Medium	Jump
13	Missing 404 Handler	 Medium	Jump
14	Navigation Bar Inconsistencies	 Medium	Jump
15	Notification Close Button Non-Functional	 Low	Jump
16		 Low	Jump

#	Bug	Priority	Section
	Grammar Error — “1 accounts”		
17	Missing Profile Features	● Low	Jump
18	Version Branding Inconsistencies	● Low	Jump

CRITICAL BUGS (Priority 1)

Bug 1: Broken Access Control

Description

Users with the **Viewer** role can access restricted admin pages (`/users` , `/audit` , `/billing` , `/regulatory-dashboard`) by typing the URL directly into the browser. The sidebar correctly hides these links, but there is no server-side or route-level guard preventing navigation.

Root Cause

Role-based access control (RBAC) is only enforced at the **UI/sidebar rendering** level (conditional rendering of `<NavLink>` components). There is no route-level middleware or wrapper component that checks the user's role before rendering the page content. The API endpoints behind these pages may also lack authorization checks.

Files to Modify

```
src/middleware.ts (or middleware.js  Next.js middleware)
src/components/guards/RoleGuard.tsx (NEW FILE  create this)
src/app/(dashboard)/users/page.tsx
src/app/(dashboard)/audit/page.tsx
src/app/(dashboard)/billing/page.tsx
src/app/(dashboard)/regulatory-dashboard/page.tsx
src/lib/auth.ts (or wherever role definitions live)
src/app/api/users/route.ts (API route protection)
src/app/api/audit/route.ts
src/app/api/billing/route.ts
```

Step-by-Step Fix

Step 1 — Define Role Permissions Map

Open (or create) `src/lib/permissions.ts` :

```
// src/lib/permissions.ts

export type UserRole = "admin" | "manager" | "analyst" | "viewer";

// Map each protected route to the roles that CAN access it
export const ROUTE_PERMISSIONS: Record<string, UserRole[]> = {
  "/users": ["admin"],
  "/audit": ["admin", "manager"],
  "/billing": ["admin", "manager"],
  "/regulatory-dashboard": ["admin", "manager", "analyst"],
  // Add more protected routes as needed
};

export function canAccessRoute(role: UserRole, path: string): boolean {
  const allowedRoles = ROUTE_PERMISSIONS[path];
  // If no rule defined, allow by default (public dashboard pages)
  if (!allowedRoles) return true;
  return allowedRoles.includes(role);
}
```

Step 2 — Create a Client-Side RoleGuard Component

Create `src/components/guards/RoleGuard.tsx` :

```
// src/components/guards/RoleGuard.tsx
"use client";

import { useEffect } from "react";
import { useRouter, usePathname } from "next/navigation";
import { useAuth } from "@hooks/useAuth"; // adjust to your auth hook
import { canAccessRoute } from "@lib/permissions";

interface RoleGuardProps {
  children: React.ReactNode;
  allowedRoles: string[];
}

export default function RoleGuard({ children, allowedRoles }: RoleGuardProps) {
  const router = useRouter();
  const pathname = usePathname();
  const { user, isLoading } = useAuth();

  useEffect(() => {
    if (!isLoading && user) {
      if (!allowedRoles.includes(user.role)) {
        // Redirect unauthorized users to the main dashboard
        router.replace("/dashboard");
      }
    }
  }, [user, isLoading, allowedRoles, router]);

  if (isLoading) {
    return <div className="flex items-center justify-center h-screen">Loading...</div>;
  }

  if (!user || !allowedRoles.includes(user.role)) {
    return null; // Render nothing while redirecting
  }

  return <>{children}</>;
}
```

Step 3 — Wrap Each Protected Page

Edit each restricted page to use the guard. Example for `/users` :

```
// src/app/(dashboard)/users/page.tsx
import RoleGuard from "@components/guards/RoleGuard";
import UsersPageContent from "@components/users/UsersPageContent";

export default function UsersPage() {
  return (
    <RoleGuard allowedRoles={["admin"]}>
      <UsersPageContent />
    </RoleGuard>
  );
}
```

Repeat for each restricted page:

Page File	allowedRoles
audit/page.tsx	["admin", "manager"]
billing/page.tsx	["admin", "manager"]
regulatory-dashboard/page.tsx	["admin", "manager", "analyst"]

Step 4 — Add Next.js Middleware (Server-Side Enforcement)

This is the **critical second layer**. Client-side guards can be bypassed; middleware cannot.

Edit `src/middleware.ts`:

```
// src/middleware.ts
import { NextRequest, NextResponse } from "next/server";
import { getToken } from "next-auth/jwt"; // adjust for your auth library
import { ROUTE_PERMISSIONS, canAccessRoute } from "@lib/permissions";

export async function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;

  // Check if this is a protected route
  const protectedRoute = Object.keys(ROUTE_PERMISSIONS).find(
    (route) => pathname === route || pathname.startsWith(route + "/")
  );

  if (protectedRoute) {
    const token = await getToken({
      req: request,
      secret: process.env.NEXTAUTH_SECRET,
    });

    if (!token) {
      // Not authenticated → redirect to login
      return NextResponse.redirect(new URL("/login", request.url));
    }

    const userRole = (token.role as string) || "viewer";

    if (!canAccessRoute(userRole as any, protectedRoute)) {
      // Authenticated but unauthorized → redirect to dashboard
      return NextResponse.redirect(new URL("/dashboard", request.url));
    }
  }

  return NextResponse.next();
}

export const config = {
  matcher: ["/users/:path*", "/audit/:path*", "/billing/:path*", "/regulatory-dash-
board/:path*"],
};
```

Step 5 — Protect API Routes

Every API route behind these pages must also check roles. Example:

```
// src/app/api/users/route.ts
import { getServerSession } from "next-auth";
import { authOptions } from "@lib/auth";
import { NextResponse } from "next/server";

export async function GET(request: Request) {
  const session = await getServerSession(authOptions);

  if (!session || session.user.role !== "admin") {
    return NextResponse.json(
      { error: "Forbidden: Insufficient permissions" },
      { status: 403 }
    );
  }

  // ... existing logic to fetch users
}
```

Testing Steps

1. Log in as a **Viewer** role user.
2. Attempt to navigate directly to `/users` → Should redirect to `/dashboard`.
3. Attempt `/audit`, `/billing`, `/regulatory-dashboard` → All should redirect.
4. Open browser DevTools → Network tab → Manually call `GET /api/users` → Should return `403`.
5. Log in as **Admin** → All pages should load normally.
6. Log in as **Manager** → `/users` blocked, `/audit` and `/billing` accessible.

Bug 2: XSS Vulnerability in User Name Field

Description

The system accepts and potentially renders raw HTML/JavaScript in the user name field. Entering `<script>alert('xss')</script>` as a name stores and displays it without sanitization, creating a **Stored XSS** vulnerability.

Root Cause

1. **No input validation** on the API endpoint that accepts name updates.
2. **No output sanitization/escaping** when rendering the user's name in the UI.
3. React's JSX auto-escapes by default, but if `dangerouslySetInnerHTML` is used anywhere, or if the name is injected into non-React contexts (emails, PDF exports, etc.), the payload executes.

Files to Modify

<code>src/app/api/users/route.ts</code>	(or profile update endpoint)
<code>src/lib/validation.ts</code>	(NEW FILE or add to existing)
<code>src/components/profile/ProfileForm.tsx</code>	(client-side validation)
<code>src/lib/sanitize.ts</code>	(NEW FILE – server-side sanitizer)

Step-by-Step Fix

Step 1 — Install DOMPurify for Server-Side Sanitization

In the Replit **Shell** tab:


```
npm install isomorphic-dompurify
```

Step 2 — Create a Sanitization Utility

Create `src/lib/sanitize.ts` :

```
// src/lib/sanitize.ts
import DOMPurify from "isomorphic-dompurify";

/**
 * Strip ALL HTML tags from a string. Used for plain-text fields
 * like names, titles, etc. that should never contain markup.
 */
export function sanitizeText(input: string): string {
  if (!input) return "";
  // DOMPurify with ALLOWED_TAGS=[] strips everything
  return DOMPurify.sanitize(input, { ALLOWED_TAGS: [] }).trim();
}

/**
 * Validate a name field: no HTML, reasonable length, only valid characters.
 */
export function validateName(name: string): { valid: boolean; error?: string } {
  if (!name || name.trim().length === 0) {
    return { valid: false, error: "Name is required" };
  }

  // Check for HTML/script tags
  const htmlPattern = /<[>]*>/g;
  if (htmlPattern.test(name)) {
    return { valid: false, error: "Name cannot contain HTML tags" };
  }

  // Length check
  if (name.trim().length < 2 || name.trim().length > 100) {
    return { valid: false, error: "Name must be between 2 and 100 characters" };
  }

  // Allow letters, spaces, hyphens, apostrophes, accented characters (African names)
  const namePattern = /^[p{L}\p{M}'\-\s.]+$/u;
  if (!namePattern.test(name.trim())) {
    return { valid: false, error: "Name contains invalid characters" };
  }

  return { valid: true };
}
```

Step 3 — Apply Server-Side Validation on the API

Edit your user/profile update API route:

```
// src/app/api/users/route.ts (or src/app/api/profile/route.ts)
import { sanitizeText, validateName } from "@lib/sanitize";

export async function PUT(request: Request) {
  const body = await request.json();

  // Sanitize and validate the name
  const sanitizedName = sanitizeText(body.name);
  const nameValidation = validateName(sanitizedName);

  if (!nameValidation.valid) {
    return NextResponse.json(
      { error: nameValidation.error },
      { status: 400 }
    );
  }

  // Use sanitizedName for the database update, NOT body.name
  const updatedUser = await prisma.user.update({
    where: { id: session.user.id },
    data: {
      name: sanitizedName,
      // ... other fields (sanitize each one)
    },
  });

  return NextResponse.json(updatedUser);
}
```

Step 4 — Add Client-Side Validation on the Form

Edit `src/components/profile/ProfileForm.tsx` :

```
// Inside ProfileForm component
const [nameError, setNameError] = useState<string>("");

const handleNameChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  const value = e.target.value;

  // Real-time check for HTML tags
  const htmlPattern = /<[>]*>/g;
  if (htmlPattern.test(value)) {
    setNameError("HTML tags are not allowed in the name field");
    return; // Don't update state with malicious input
  }

  setNameError("");
  setFormData((prev) => ({ ...prev, name: value }));
};

// In JSX:
<div>
  <label htmlFor="name" className="block text-sm font-medium text-gray-700">
    Full Name
  </label>
  <input
    id="name"
    type="text"
    value={formData.name}
    onChange={handleNameChange}
    maxLength={100}
    className={`mt-1 block w-full rounded-md border ${
      nameError ? "border-red-500" : "border-gray-300"
    } px-3 py-2`}
  />
  {nameError && (
    <p className="mt-1 text-sm text-red-600">{nameError}</p>
  )}
</div>
```

Step 5 — Audit All Rendering Points

Search your entire codebase for anywhere the user name is rendered. In Replit Shell:

```
grep -rn "dangerouslySetInnerHTML" src/
grep -rn "user\.name\|user\.fullName\|userName" src/components/ src/app/
```

Ensure **every** rendering of user.name uses plain JSX (which auto-escapes):

```
// ✅ SAFE – React auto-escapes
<span>{user.name}</span>

// ❌ DANGEROUS – never do this with user input
<span dangerouslySetInnerHTML={{ __html: user.name }} />
```

Step 6 — Clean Existing Malicious Data in Database

Run a one-time database migration/script:

```
-- In your PostgreSQL console or via a migration
UPDATE users
SET name = regexp_replace(name, '<[^>]*>', '', 'g')
WHERE name ~ '<[^>]*>';
```

Testing Steps

1. Go to **Profile Edit** → Enter `<script>alert('xss')</script>` as name.
2. The form should show an error message and **reject** the input.
3. Try submitting via API directly (Postman/curl) with the script tag → Should return `400`.
4. Verify the user's name displays as plain text everywhere (header, profile, audit logs).
5. Check that legitimate African names with accents/apostrophes still work: `N'Guessan`, `Müller`, `Olúwafẹ́mí`.

Bug 3: Portfolio Excel Export Fails

Description

Clicking “Export to Excel” on the portfolio page fails silently. The browser either downloads a broken file, downloads nothing, or produces a file with a UUID-based filename (e.g., `a3f8b2c1-...xlsx`) instead of a human-readable name. No error message is shown to the user.

Root Cause

1. The export function likely passes a raw database record ID (UUID) as the filename.
2. The Excel generation library (e.g., `xlsx`, `exceljs`) may be receiving improperly formatted data.
3. The `Content-Disposition` header may be malformed or missing.
4. Column definitions between the data query and the Excel writer are mismatched.

Files to Modify

<code>src/app/api/portfolio/export/route.ts</code>	(API endpoint for export)
<code>src/lib/export/excelExport.ts</code>	(Excel generation utility)
<code>src/components/portfolio/ExportButton.tsx</code>	(Client-side trigger)

Step-by-Step Fix

Step 1 — Fix the API Export Endpoint

```
// src/app/api/portfolio/export/route.ts
import { NextResponse } from "next/server";
import { getServerSession } from "next-auth";
import ExcelJS from "exceljs";
import { prisma } from "@/lib/prisma";

export async function GET(request: Request) {
  try {
    const session = await getServerSession();
    if (!session) {
      return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
    }

    const { searchParams } = new URL(request.url);
    const portfolioId = searchParams.get("portfolioId");

    // Fetch portfolio data
    const portfolio = await prisma.portfolio.findUnique({
      where: { id: portfolioId! },
      include: {
        loans: {
          include: {
            borrower: true,
          },
        },
      },
    });

    if (!portfolio) {
      return NextResponse.json({ error: "Portfolio not found" }, { status: 404 });
    }

    // Create workbook
    const workbook = new ExcelJS.Workbook();
    workbook.creator = "Africa Credit Hub";
    workbook.created = new Date();

    const worksheet = workbook.addWorksheet("Portfolio Data");

    // Define columns explicitly – THIS is usually where the mismatch occurs
    worksheet.columns = [
      { header: "Loan ID", key: "loanId", width: 20 },
      { header: "Borrower Name", key: "borrowerName", width: 25 },
      { header: "Borrower ID", key: "borrowerId", width: 20 },
      { header: "Principal Amount", key: "principal", width: 18 },
      { header: "Interest Rate (%)", key: "interestRate", width: 16 },
      { header: "Term (Months)", key: "term", width: 14 },
      { header: "Status", key: "status", width: 14 },
      { header: "Disbursement Date", key: "disbursementDate", width: 18 },
      { header: "Maturity Date", key: "maturityDate", width: 18 },
      { header: "Outstanding Balance", key: "outstandingBalance", width: 20 },
    ];

    // Add data rows
    portfolio.loans.forEach((loan) => {
      worksheet.addRow({
        loanId: loan.id,
        borrowerName: loan.borrower?.name || "N/A",
        borrowerId: loan.borrower?.id || "N/A",
        principal: loan.principalAmount,
        interestRate: loan.interestRate,
        term: loan.termMonths,
      });
    });
  }
}
```

```

        status: loan.status,
        disbursementDate: loan.disbursementDate
          ? new Date(loan.disbursementDate).toISOString().split("T")[0]
          : "N/A",
        maturityDate: loan.maturityDate
          ? new Date(loan.maturityDate).toISOString().split("T")[0]
          : "N/A",
        outstandingBalance: loan.outstandingBalance,
      });
    });

    // Style header row
    worksheet.getRow(1).font = { bold: true };
    worksheet.getRow(1).fill = {
      type: "pattern",
      pattern: "solid",
      fgColor: { argb: "FF1A5632" }, // Africa Credit Hub green
    };
    worksheet.getRow(1).font = { bold: true, color: { argb: "FFFFFF" } };

    // Generate buffer
    const buffer = await workbook.xlsx.writeBuffer();

    // ✅ FIX: Create human-readable filename with date
    const dateStr = new Date().toISOString().split("T")[0];
    const safeName = (portfolio.name || "portfolio")
      .replace(/[^a-zA-Z0-9_-]/g, "_")
      .substring(0, 50);
    const filename = `${safeName}_export_${dateStr}.xlsx`;

    // ✅ FIX: Set correct headers
    return new NextResponse(buffer, {
      status: 200,
      headers: {
        "Content-Type":
          "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
        "Content-Disposition": `attachment; filename="${filename}"`,
        "Content-Length": buffer.byteLength.toString(),
      },
    });
  } catch (error) {
    console.error("Portfolio export error:", error);
    return NextResponse.json(
      { error: "Failed to generate export. Please try again." },
      { status: 500 }
    );
  }
}

```

Step 2 — Fix the Client-Side Export Button


```
// src/components/portfolio/ExportButton.tsx
"use client";

import { useState } from "react";
import { Download, Loader2 } from "lucide-react";
import { useTranslation } from "next-i18next";

interface ExportButtonProps {
  portfolioId: string;
}

export default function ExportButton({ portfolioId }: ExportButtonProps) {
  const [isExporting, setIsExporting] = useState(false);
  const [error, setError] = useState<string | null>(null);
  const { t } = useTranslation();

  const handleExport = async () => {
    setIsExporting(true);
    setError(null);

    try {
      const response = await fetch(
        `/api/portfolio/export?portfolioId=${portfolioId}`
      );

      if (!response.ok) {
        const errData = await response.json().catch(() => ({}));
        throw new Error(errData.error || "Export failed");
      }

      // ✅ FIX: Extract filename from Content-Disposition header
      const contentDisposition = response.headers.get("Content-Disposition");
      let filename = "portfolio_export.xlsx"; // fallback
      if (contentDisposition) {
        const match = contentDisposition.match(/filename="?([^";\\n]+)"?/);
        if (match && match[1]) {
          filename = match[1];
        }
      }

      const blob = await response.blob();
      const url = window.URL.createObjectURL(blob);
      const a = document.createElement("a");
      a.href = url;
      a.download = filename;
      document.body.appendChild(a);
      a.click();

      // Cleanup
      window.URL.revokeObjectURL(url);
      document.body.removeChild(a);
    } catch (err: any) {
      console.error("Export error:", err);
      setError(err.message || "Failed to export portfolio. Please try again.");
    } finally {
      setIsExporting(false);
    }
  };

  return (
    <div>
      <button
```

```

onClick={handleExport}
disabled={isExporting}
className="inline-flex items-center gap-2 rounded-md bg-green-700 px-4 py-2
          text-sm font-medium text-white hover:bg-green-800
          disabled:cursor-not-allowed disabled:opacity-50"
>
  {isExporting ? (
    <Loader2 className="h-4 w-4 animate-spin" />
  ) : (
    <Download className="h-4 w-4" />
  )}
  {isExporting ? t("common.exporting") : t("common.exportExcel")}
</button>

{/* ☒ FIX: Show error to user instead of failing silently */}
{error && (
  <p className="mt-2 text-sm text-red-600">{error}</p>
)}
</div>
);
}

```

Testing Steps

1. Navigate to **Portfolio** page with data loaded.
2. Click **Export to Excel** → File should download with a name like `my_portfolio_export_2026-04-02.xlsx`.
3. Open the file in Excel/Google Sheets → All columns should be present and data intact.
4. Test with an empty portfolio → Should either export headers only or show a “No data to export” message.
5. Check browser DevTools Network tab → Response should have correct `Content-Type` and `Content-Disposition`.

Bug 4: Credit Report Search Non-Functional

Description

The credit report search feature cannot find any borrowers, even when they exist in the database. The search field accepts input but returns zero results for every query.

Root Cause

Likely one or more of:

1. The search API query uses an exact match (`=`) instead of a partial/case-insensitive match (`ILIKE`).
2. The search field sends the query to the wrong API endpoint or parameter name.
3. The API filters on a field that is empty/null (e.g., searching `nationalId` when the data only has `borrowerId`).
4. A missing or incorrect `WHERE` clause in the database query.

Files to Modify

```

src/app/api/credit-reports/search/route.ts  (API search endpoint)
src/components/credit/CreditReportSearch.tsx (Search form component)
src/lib/queries/creditQueries.ts            (Database query helpers)

```

Step-by-Step Fix

Step 1 — Fix the API Search Endpoint

```
// src/app/api/credit-reports/search/route.ts
import { NextResponse } from "next/server";
import { prisma } from "@lib/prisma";
import { getServerSession } from "next-auth";

export async function GET(request: Request) {
  try {
    const session = await getServerSession();
    if (!session) {
      return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
    }

    const { searchParams } = new URL(request.url);
    const query = searchParams.get("q")?.trim();
    // ✅ FIX: Also check alternate parameter names the frontend might send
    const search = query || searchParams.get("search")?.trim() || searchParams.get("query")?.trim();

    if (!search || search.length < 2) {
      return NextResponse.json(
        { error: "Search query must be at least 2 characters", results: [] },
        { status: 400 }
      );
    }

    // ✅ FIX: Use case-insensitive partial matching across multiple fields
    const borrowers = await prisma.borrower.findMany({
      where: {
        OR: [
          { name: { contains: search, mode: "insensitive" } },
          { email: { contains: search, mode: "insensitive" } },
          { nationalId: { contains: search, mode: "insensitive" } },
          { phoneNumber: { contains: search, mode: "insensitive" } },
          // ✅ FIX: Also search by borrower reference/account number
          { referenceNumber: { contains: search, mode: "insensitive" } },
        ],
      },
      include: {
        creditReports: {
          orderBy: { createdAt: "desc" },
          take: 1, // latest report only
        },
      },
      take: 20, // Limit results for performance
    });

    return NextResponse.json({
      results: borrowers,
      count: borrowers.length,
    });
  } catch (error) {
    console.error("Credit report search error:", error);
    return NextResponse.json(
      { error: "Search failed. Please try again." },
      { status: 500 }
    );
  }
}
```

If using raw SQL instead of Prisma:

```
// Alternative: Raw SQL approach
const borrowers = await prisma.$queryRaw`
  SELECT b.*, cr.score as latest_score, cr.created_at as report_date
  FROM borrowers b
  LEFT JOIN LATERAL (
    SELECT score, created_at
    FROM credit_reports
    WHERE borrower_id = b.id
    ORDER BY created_at DESC
    LIMIT 1
  ) cr ON true
  WHERE
    b.name ILIKE ${'%' + search + '%'}
    OR b.email ILIKE ${'%' + search + '%'}
    OR b.national_id ILIKE ${'%' + search + '%'}
    OR b.phone_number ILIKE ${'%' + search + '%'}
    OR b.reference_number ILIKE ${'%' + search + '%'}
  LIMIT 20
`;
```

Step 2 — Fix the Search Form Component

```
// src/components/credit/CreditReportSearch.tsx
"use client";

import { useState, useCallback } from "react";
import { Search, Loader2, AlertCircle } from "lucide-react";
import { useTranslation } from "next-i18next";
import debounce from "lodash/debounce";

export default function CreditReportSearch() {
  const [searchTerm, setSearchTerm] = useState("");
  const [results, setResults] = useState<any[]>([]);
  const [isSearching, setIsSearching] = useState(false);
  const [error, setError] = useState<string | null>(null);
  const [hasSearched, setHasSearched] = useState(false);
  const { t } = useTranslation();

  const performSearch = useCallback(
    debounce(async (query: string) => {
      if (query.trim().length < 2) {
        setResults([]);
        setHasSearched(false);
        return;
      }

      setIsSearching(true);
      setError(null);

      try {
        // ✓ FIX: Ensure correct parameter name matches the API
        const response = await fetch(
          `/api/credit-reports/search?q=${encodeURIComponent(query.trim())}`
        );

        if (!response.ok) {
          const errData = await response.json().catch(() => ({}));
          throw new Error(errData.error || "Search failed");
        }

        const data = await response.json();
        setResults(data.results || []);
        setHasSearched(true);
      } catch (err: any) {
        console.error("Search error:", err);
        setError(err.message || "Search failed. Please try again.");
        setResults([]);
      } finally {
        setIsSearching(false);
      }
    }, 300),
    []
  );

  const handleInputChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    const value = e.target.value;
    setSearchTerm(value);
    performSearch(value);
  };

  return (
    <div className="w-full max-w-2xl">
      <div className="relative">
```

```

<Search className="absolute left-3 top-1/2 h-5 w-5 -translate-y-1/2 text-
gray-400" />
<input
  type="text"
  value={searchTerm}
  onChange={handleInputChange}
  placeholder={t("credit.searchPlaceholder", "Search by name, ID, email, or
phone...")}
  className="w-full rounded-lg border border-gray-300 py-3 pl-10 pr-4
text-sm focus:border-green-500 focus:ring-2 focus:ring-green-200"
/>
{isSearching && (
  <Loader2 className="absolute right-3 top-1/2 h-5 w-5 -translate-y-1/2
animate-spin text-gray-400" />
)}
</div>

{/* Error Message */}
{error && (
  <div className="mt-3 flex items-center gap-2 rounded-md bg-red-50 p-3 text-sm
text-red-700">
    <AlertCircle className="h-4 w-4" />
    {error}
  </div>
)}

{/* Results */}
{hasSearched && results.length === 0 && !isSearching && !error && (
  <p className="mt-4 text-center text-sm text-gray-500">
    {t("credit.noResults", "No borrowers found matching your search.")}
  </p>
)}

{results.length > 0 && (
  <div className="mt-4 space-y-2">
    {results.map((borrower) => (
      <div
        key={borrower.id}
        className="cursor-pointer rounded-lg border border-gray-200 p-4
transition hover:border-green-300 hover:bg-green-50"
        onClick={() => {
          // Navigate to credit report details
          window.location.href = `/credit-reports/${borrower.id}`;
        }}
      >
        <p className="font-medium text-gray-900">{borrower.name}</p>
        <p className="text-sm text-gray-500">
          {borrower.nationalId || borrower.referenceNumber} {borrower.email}
        </p>
        {borrower.creditReports?.[0] && (
          <p className="mt-1 text-sm text-green-700">
            {t("credit.latestScore")}: {borrower.creditReports[0].score}
          </p>
        )}
      </div>
    )}
  </div>
)}
</div>
);
}

```


Testing Steps

1. Navigate to **Credit Reports** → Search section.
2. Type a known borrower name (e.g., "Amina") → Results should appear after 2+ characters.
3. Search by national ID → Should find the matching borrower.
4. Search by email → Should find the matching borrower.
5. Search a non-existent name → Should show "No borrowers found" message.
6. Check the Network tab → API call uses `?q=...` and returns proper JSON.

HIGH PRIORITY BUGS (Priority 2)

Bug 5: Sidebar Navigation Not Translated

Description

The sidebar navigation links (Dashboard, Borrowers, Portfolios, Credit Reports, etc.) remain in English regardless of the selected language (French, Portuguese, Arabic, Swahili).

Root Cause

The sidebar component uses **hardcoded English strings** instead of translation keys from the i18n system. The navigation items are likely defined as a static array with English labels.

Files to Modify

<code>src/components/layout/Sidebar.tsx</code>	(Main sidebar component)
<code>public/locales/en/common.json</code>	(English translations)
<code>public/locales/fr/common.json</code>	(French translations)
<code>public/locales/pt/common.json</code>	(Portuguese translations)
<code>public/locales/ar/common.json</code>	(Arabic translations)
<code>public/locales/sw/common.json</code>	(Swahili translations)

Step-by-Step Fix

Step 1 — Add Navigation Translation Keys to All Locale Files

English (`public/locales/en/common.json`) — add under a `"nav"` block:

```
{
  "nav": {
    "dashboard": "Dashboard",
    "borrowers": "Borrowers",
    "portfolios": "Portfolios",
    "creditReports": "Credit Reports",
    "blockchain": "Blockchain Ledger",
    "users": "User Management",
    "audit": "Audit Log",
    "billing": "Billing",
    "regulatory": "Regulatory Dashboard",
    "settings": "Settings",
    "profile": "Profile",
    "logout": "Logout"
  }
}
```

French (public/locales/fr/common.json):

```
{
  "nav": {
    "dashboard": "Tableau de bord",
    "borrowers": "Emprunteurs",
    "portfolios": "Portefeuilles",
    "creditReports": "Rapports de crédit",
    "blockchain": "Registre Blockchain",
    "users": "Gestion des utilisateurs",
    "audit": "Journal d'audit",
    "billing": "Facturation",
    "regulatory": "Tableau réglementaire",
    "settings": "Paramètres",
    "profile": "Profil",
    "logout": "Déconnexion"
  }
}
```

Portuguese (public/locales/pt/common.json):

```
{
  "nav": {
    "dashboard": "Painel",
    "borrowers": "Mutuários",
    "portfolios": "Carteiras",
    "creditReports": "Relatórios de Crédito",
    "blockchain": "Livro Blockchain",
    "users": "Gestão de Utilizadores",
    "audit": "Registo de Auditoria",
    "billing": "Faturação",
    "regulatory": "Painel Regulatório",
    "settings": "Configurações",
    "profile": "Perfil",
    "logout": "Sair"
  }
}
```

Arabic (public/locales/ar/common.json):

```
{
  "nav": {
    "dashboard": "لوحة التحكم",
    "borrowers": "المقرضون",
    "portfolios": "المحافظ",
    "creditReports": "تقارير الائتمان",
    "blockchain": "سجل البلوكتشين",
    "users": "إدارة المستخدمين",
    "audit": "سجل التدقيق",
    "billing": "الفوترة",
    "regulatory": "لوحة الرقابة",
    "settings": "الإعدادات",
    "profile": "الملف الشخصي",
    "logout": "تسجيل الخروج"
  }
}
```

Swahili (public/locales/sw/common.json):

```
{
  "nav": {
    "dashboard": "Dashibodi",
    "borrowers": "Wakopaji",
    "portfolios": "Kanda za Mikopo",
    "creditReports": "Ripoti za Mikopo",
    "blockchain": "Daftari la Blockchain",
    "users": "Usimamizi wa Watumiaji",
    "audit": "Kumbukumbu za Ukaguzi",
    "billing": "Malipo",
    "regulatory": "Dashibodi ya Udhibiti",
    "settings": "Mipangilio",
    "profile": "Wasifu",
    "logout": "Ondoka"
  }
}
```

Step 2 — Refactor the Sidebar Component

```
// src/components/layout/Sidebar.tsx
"use client";

import { useTranslation } from "next-i18next";
import { usePathname } from "next/navigation";
import Link from "next/link";
import {
  LayoutDashboard, Users, Briefcase, FileText,
  Link2, Shield, ScrollText, CreditCard, BarChart3,
  Settings, User, LogOut,
} from "lucide-react";
import { useAuth } from "@hooks/useAuth";

export default function Sidebar() {
  const { t } = useTranslation("common");
  const pathname = usePathname();
  const { user } = useAuth();

  // ✅ FIX: Use translation keys instead of hardcoded strings
  const navItems = [
    { href: "/dashboard", icon: LayoutDashboard, label: t("nav.dashboard"), roles: ["admin", "manager", "analyst", "viewer"] },
    { href: "/borrowers", icon: Users, label: t("nav.borrowers"), roles: ["admin", "manager", "analyst", "viewer"] },
    { href: "/portfolios", icon: Briefcase, label: t("nav.portfolios"), roles: ["admin", "manager", "analyst", "viewer"] },
    { href: "/credit-reports", icon: FileText, label: t("nav.creditReports"), roles: ["admin", "manager", "analyst", "viewer"] },
    { href: "/blockchain", icon: Link2, label: t("nav.blockchain"), roles: ["admin", "manager", "analyst", "viewer"] },
    { href: "/users", icon: Shield, label: t("nav.users"), roles: ["admin"] },
    { href: "/audit", icon: ScrollText, label: t("nav.audit"), roles: ["admin", "manager"] },
    { href: "/billing", icon: CreditCard, label: t("nav.billing"), roles: ["admin", "manager"] },
    { href: "/regulatory-dashboard", icon: BarChart3, label: t("nav.regulatory"), roles: ["admin", "manager", "analyst"] },
  ];

  // Filter items based on user role
  const visibleItems = navItems.filter((item) =>
    item.roles.includes(user?.role || "viewer")
  );

  return (
    <aside className="flex h-screen w-64 flex-col bg-gray-900 text-white" dir="auto">
      <div className="flex h-16 items-center px-6">
        <h1 className="text-lg font-bold">Africa Credit Hub</h1>
      </div>

      <div className="flex-1 space-y-1 px-3 py-4">
        {visibleItems.map((item) => {
          const isActive = pathname === item.href || pathname.startsWith(item.href + "/" );

          return (
            <Link
              key={item.href}
              href={item.href}
              className={`flex items-center gap-3 rounded-lg px-3 py-2.5 text-sm font-`

```

```

medium transition
    ${isActive
      ? "bg-green-700 text-white"
      : "text-gray-300 hover:bg-gray-800 hover:text-white"
    }`
  >
    <item.icon className="h-5 w-5 flex-shrink-0" />
    <span>{item.label}</span>
  </Link>
);
}}
</nav>

{/* Bottom section */}
<div className="border-t border-gray-700 p-3 space-y-1">
  <Link href="/settings" className="flex items-center gap-3 rounded-lg px-3
py-2.5 text-sm text-gray-300 hover:bg-gray-800">
    <Settings className="h-5 w-5" />
    <span>{t("nav.settings")}</span>
  </Link>
  <Link href="/profile" className="flex items-center gap-3 rounded-lg px-3
py-2.5 text-sm text-gray-300 hover:bg-gray-800">
    <User className="h-5 w-5" />
    <span>{t("nav.profile")}</span>
  </Link>
</div>
</aside>
);
}

```

Testing Steps

1. Switch language to **French** → All sidebar items should display in French.
2. Switch to **Portuguese** → Sidebar should update to Portuguese.
3. Switch to **Arabic** → Sidebar should show Arabic text (with RTL support via `dir="auto"`).
4. Switch to **Swahili** → Sidebar should show Swahili text.
5. Switch back to **English** → Sidebar should revert to English.
6. Verify that no hardcoded English text remains in the sidebar across all languages.

Bug 6: Information Leakage on /profile and /settings

Description

Navigating to `/profile` or `/settings` returns a `404` page that displays **developer-facing error messages** such as stack traces, file paths, or component names. This leaks internal implementation details to the user.

Root Cause

1. These routes are either **not implemented** or their page files are missing/misconfigured.
2. Next.js is running in **development mode** in production (or `NODE_ENV` is not set to `production`), causing detailed error output.
3. There is no custom `not-found.tsx` or error boundary that provides a user-friendly message.

Files to Modify

src/app/(dashboard)/profile/page.tsx	(Create if missing)
src/app/(dashboard)/settings/page.tsx	(Create if missing)
src/app/not-found.tsx	(Custom 404 page)
next.config.js	(Environment configuration)

Step-by-Step Fix

Step 1 — Create Stub Pages (If Routes Don't Exist Yet)

```
// src/app/(dashboard)/profile/page.tsx
import { getSession } from "next-auth";
import { redirect } from "next/navigation";

export default async function ProfilePage() {
  const session = await getSession();
  if (!session) redirect("/login");

  return (
    <div className="mx-auto max-w-2xl p-6">
      <h1 className="text-2xl font-bold text-gray-900">Profile</h1>
      <p className="mt-2 text-gray-600">Manage your account details.</p>
      {/* Profile form component goes here – see Bug 17 for full implementation */}
      <div className="mt-6 rounded-lg border border-gray-200 bg-white p-6 shadow-sm">
        <p className="text-gray-500">Profile management coming soon.</p>
      </div>
    </div>
  );
}
```

```
// src/app/(dashboard)/settings/page.tsx
import { getSession } from "next-auth";
import { redirect } from "next/navigation";

export default async function SettingsPage() {
  const session = await getSession();
  if (!session) redirect("/login");

  return (
    <div className="mx-auto max-w-2xl p-6">
      <h1 className="text-2xl font-bold text-gray-900">Settings</h1>
      <p className="mt-2 text-gray-600">Application preferences and configuration.</p>
      <div className="mt-6 rounded-lg border border-gray-200 bg-white p-6 shadow-sm">
        <p className="text-gray-500">Settings page coming soon.</p>
      </div>
    </div>
  );
}
```

Step 2 — Create a Custom 404 Page

```
// src/app/not-found.tsx
import Link from "next/link";

export default function NotFound() {
  return (
    <div className="flex min-h-screen flex-col items-center justify-center bg-gray-50 px-4">
      <div className="text-center">
        <h1 className="text-6xl font-bold text-gray-900">404</h1>
        <p className="mt-4 text-xl text-gray-600">
          The page you're looking for doesn't exist.
        </p>
        <p className="mt-2 text-gray-500">
          It may have been moved or the URL might be incorrect.
        </p>
        <div className="mt-8 flex gap-4 justify-center">
          <Link
            href="/dashboard"
            className="rounded-md bg-green-700 px-6 py-3 text-sm font-medium text-white hover:bg-green-800"
          >
            Go to Dashboard
          </Link>
          <Link
            href="/"
            className="rounded-md border border-gray-300 px-6 py-3 text-sm font-medium text-gray-700 hover:bg-gray-50"
          >
            Go Home
          </Link>
        </div>
      </div>
    </div>
  );
}
```

Step 3 — Ensure Production Mode Hides Errors

Check `next.config.js` :

```
// next.config.js
/** @type {import('next').NextConfig} */
const nextConfig = {
  // ✅ Ensure production builds don't leak error details
  productionBrowserSourceMaps: false,

  // ✅ Custom error handling
  async redirects() {
    return [];
    // Do NOT add catch-all redirects here – use not-found.tsx instead
  },
};

module.exports = nextConfig;
```

In Replit's `.replit` or environment settings, ensure:


```
NODE_ENV=production
```

Testing Steps

1. Navigate to `/profile` → Should show the profile page (or “coming soon” stub), NOT a 404 error.
2. Navigate to `/settings` → Should show the settings page, NOT a 404 error.
3. Navigate to `/this-page-does-not-exist` → Should show the custom 404 page with navigation links.
4. Verify that **no stack traces**, file paths, or component names appear on any error page.
5. View page source on the 404 page → Confirm no internal details are leaked.

Bug 7: Broken Terms/Privacy Links

Description

Clicking “Terms of Service” or “Privacy Policy” links (typically in the footer or login page) redirects users to `/security` instead of the correct legal pages (`/terms` and `/privacy`).

Root Cause

The `href` values for these links are **incorrectly set to** `/security` — likely a copy-paste error or a placeholder that was never updated.

Files to Modify

<code>src/components/layout/Footer.tsx</code>	(Footer links)
<code>src/app/(auth)/login/page.tsx</code>	(Login page footer links)
<code>src/app/(auth)/register/page.tsx</code>	(Registration page links)
<code>src/app/terms/page.tsx</code>	(Create if missing)
<code>src/app/privacy/page.tsx</code>	(Create if missing)

Step-by-Step Fix

Step 1 — Search and Fix All Incorrect Links

In Replit Shell, find all instances:

```
grep -rn "/security" src/components/ src/app/
```

Replace every instance where `/security` is used for terms or privacy:

```
// ❌ WRONG
<Link href="/security">Terms of Service</Link>
<Link href="/security">Privacy Policy</Link>

// ✅ CORRECT
<Link href="/terms">Terms of Service</Link>
<Link href="/privacy">Privacy Policy</Link>
```

Step 2 — Fix the Footer Component

```
// src/components/layout/Footer.tsx
import Link from "next/link";
import { useTranslation } from "next-i18next";

export default function Footer() {
  const { t } = useTranslation("common");
  const currentYear = new Date().getFullYear();

  return (
    <footer className="border-t border-gray-200 bg-white px-6 py-4">
      <div className="flex flex-col items-center justify-between gap-2 sm:flex-row">
        <p className="text-sm text-gray-500">
          © {currentYear} Africa Credit Hub. {t("footer.allRightsReserved", "All
rights reserved.")}
        </p>
        <div className="flex gap-6">
          {/* ✅ FIX: Correct href values */}
          <Link
            href="/terms"
            className="text-sm text-gray-500 hover:text-green-700 hover:underline"
          >
            {t("footer.terms", "Terms of Service")}
          </Link>
          <Link
            href="/privacy"
            className="text-sm text-gray-500 hover:text-green-700 hover:underline"
          >
            {t("footer.privacy", "Privacy Policy")}
          </Link>
          <Link
            href="/security"
            className="text-sm text-gray-500 hover:text-green-700 hover:underline"
          >
            {t("footer.security", "Security")}
          </Link>
        </div>
      </div>
    </footer>
  );
}
```

Step 3 — Create Terms and Privacy Pages (If Missing)

```
// src/app/terms/page.tsx
export default function TermsPage() {
  return (
    <div className="mx-auto max-w-4xl px-6 py-12">
      <h1 className="text-3xl font-bold text-gray-900">Terms of Service</h1>
      <p className="mt-2 text-sm text-gray-500">Last updated: April 2026</p>

      <div className="prose mt-8 max-w-none text-gray-700">
        <h2>1. Acceptance of Terms</h2>
        <p>
          By accessing and using Africa Credit Hub ("the Platform"), you agree
          to be bound by these Terms of Service. If you do not agree, please
          discontinue use immediately.
        </p>

        <h2>2. Description of Service</h2>
        <p>
          Africa Credit Hub provides credit scoring, portfolio management, and
          regulatory reporting services for financial institutions operating
          across the African continent.
        </p>

        <h2>3. User Responsibilities</h2>
        <p>
          Users are responsible for maintaining the confidentiality of their
          account credentials and for all activity that occurs under their
          account.
        </p>

        { /* Add remaining legal sections as appropriate */ }
        <h2>4. Data Protection</h2>
        <p>
          We comply with applicable data protection regulations including but
          not limited to the African Union Convention on Cyber Security and
          Personal Data Protection (Malabo Convention) and relevant national
          data protection laws.
        </p>

        <h2>5. Contact</h2>
        <p>
          For questions regarding these terms, contact us at
          legal@afriacredithub.com.
        </p>
      </div>
    </div>
  );
}
```

```
// src/app/privacy/page.tsx
export default function PrivacyPage() {
  return (
    <div className="mx-auto max-w-4xl px-6 py-12">
      <h1 className="text-3xl font-bold text-gray-900">Privacy Policy</h1>
      <p className="mt-2 text-sm text-gray-500">Last updated: April 2026</p>

      <div className="prose mt-8 max-w-none text-gray-700">
        <h2>1. Information We Collect</h2>
        <p>
          We collect information you provide directly, including name, email,
          organization details, and usage data necessary for the operation of
          credit scoring and portfolio management services.
        </p>

        <h2>2. How We Use Your Information</h2>
        <p>
          Your information is used to provide, maintain, and improve the
          Platform's services, including generating credit reports and
          regulatory filings.
        </p>

        <h2>3. Data Sharing</h2>
        <p>
          We do not sell personal data. Data may be shared with regulatory
          bodies as required by law, or with service providers who assist in
          platform operations under strict data processing agreements.
        </p>

        <h2>4. Data Retention</h2>
        <p>
          Credit data is retained in accordance with applicable financial
          regulations. Users may request data deletion subject to regulatory
          retention requirements.
        </p>

        <h2>5. Contact</h2>
        <p>
          For privacy inquiries, contact our Data Protection Officer at
          privacy@africacredithub.com.
        </p>
      </div>
    </div>
  );
}
```

Testing Steps

1. Go to the **Login page** → Click "Terms of Service" → Should navigate to `/terms`.
2. Click "Privacy Policy" → Should navigate to `/privacy`.
3. Go to the **Footer** on any page → Click "Terms" → Goes to `/terms`.
4. Click "Privacy" → Goes to `/privacy`.
5. Click "Security" → Goes to `/security` (this one should still work).
6. Verify that `/terms` and `/privacy` render content, not a 404 page.

Bug 8: Incomplete Language Translations (AR/SW)

Description

When Arabic or Swahili is selected, many UI strings fail to render — showing either blank text, raw keys (e.g., `dashboard.title`), or falling back to English inconsistently.

Root Cause

1. The Arabic and Swahili locale JSON files are **incomplete** — many keys are missing.
2. The `i18n` configuration may not have a proper **fallback chain** configured.
3. For Arabic, **RTL layout support** may also be missing, causing text rendering issues.

Files to Modify

<code>next-i18next.config.js</code>	(<code>i18n</code> configuration)
<code>public/locales/ar/common.json</code>	(Arabic translations)
<code>public/locales/sw/common.json</code>	(Swahili translations)
<code>src/app/layout.tsx</code>	(RTL support)
<code>src/components/LanguageSwitcher.tsx</code>	(Language switcher component)

Step-by-Step Fix

Step 1 — Configure `i18n` Fallback

```
// next-i18next.config.js
module.exports = {
  i18n: {
    defaultLocale: "en",
    locales: ["en", "fr", "pt", "ar", "sw"],
    // ✅ FIX: Add fallback so missing keys display English instead of raw keys
    fallbackLng: {
      ar: ["en"],
      sw: ["en"],
      fr: ["en"],
      pt: ["en"],
      default: ["en"],
    },
  },
  // ✅ FIX: Do not show raw keys to users
  returnNull: false,
  returnEmptyString: false,
},
// ✅ Load all namespaces
ns: ["common", "dashboard", "credit", "billing", "auth"],
defaultNS: "common",
};
```

Step 2 — Complete the Arabic Translation File

Run this in Replit Shell to find all translation keys used in the app:

```
grep -rohP "t\(['\"](?:[^\\\"]|\\\")+\)" src/ | sort -u > /tmp/all_keys.txt
```

Then ensure each key exists in `public/locales/ar/common.json`. Here is a comprehensive example:

```

{
  "nav": {
    "dashboard": "لوحة التحكم",
    "borrowers": "المقترضون",
    "portfolios": "المحافظ",
    "creditReports": "تقارير الائتمان",
    "blockchain": "سجل البلوكتشين",
    "users": "إدارة المستخدمين",
    "audit": "سجل التدقيق",
    "billing": "الفوترة",
    "regulatory": "لوحة الرقابة",
    "settings": "الإعدادات",
    "profile": "الملف الشخصي",
    "logout": "تسجيل الخروج"
  },
  "dashboard": {
    "title": "لوحة التحكم",
    "welcome": "مرحباً، {{name}}",
    "totalBorrowers": "إجمالي المقترضين",
    "activeLoans": "القروض النشطة",
    "totalPortfolioValue": "إجمالي قيمة المحفظة",
    "recentActivity": "النشاط الأخير"
  },
  "borrowers": {
    "title": "المقترضون",
    "addNew": "إضافة مقترض جديد",
    "search": "...البحث عن مقترض",
    "name": "الاسم",
    "email": "البريد الإلكتروني",
    "phone": "الهاتف",
    "status": "الحالة",
    "actions": "الإجراءات",
    "active": "نشط",
    "inactive": "غير نشط",
    "defaulted": "متعثر"
  },
  "credit": {
    "title": "تقارير الائتمان",
    "searchPlaceholder": "...البحث بالاسم أو الرقم الوطني أو البريد الإلكتروني",
    "noResults": "...لم يتم العثور على مقترضين مطابقين",
    "latestScore": "أحدث تقييم",
    "generateReport": "إنشاء تقرير",
    "downloadReport": "تحميل التقرير"
  },
  "billing": {
    "title": "الفوترة",
    "currentPlan": "الخطة الحالية",
    "invoices": "الفواتير",
    "paymentHistory": "سجل المدفوعات",
    "noRecordsTitle": "...لا توجد سجلات",
    "noRecordsDesc": "...لم يتم العثور على سجلات فواتير حتى الآن"
  },
  "common": {
    "save": "حفظ",
    "cancel": "إلغاء",
    "delete": "حذف",
    "edit": "تعديل",
    "export": "تصدير",
    "exportExcel": "Excel تصدير إلى",
    "exporting": "...جارٍ التصدير",
    "loading": "...جارٍ التحميل",
    "confirm": "تأكيد"
  }
}

```

```

    "actions": "الإجراءات",
    "search": "بحث",
    "filter": "تصفية",
    "noData": "لا توجد بيانات",
    "error": "خطأ",
    "success": "تم بنجاح"
  },
  "auth": {
    "login": "تسجيل الدخول",
    "register": "إنشاء حساب",
    "email": "البريد الإلكتروني",
    "password": "كلمة المرور",
    "forgotPassword": "نسيت كلمة المرور؟",
    "rememberMe": "تذكرني"
  },
  "footer": {
    "allRightsReserved": "جميع الحقوق محفوظة.",
    "terms": "شروط الخدمة",
    "privacy": "سياسة الخصوصية",
    "security": "الأمان"
  }
}

```

Step 3 — Complete the Swahili Translation File

Apply the same process for `public/locales/sw/common.json` :

```

{
  "nav": {
    "dashboard": "Dashibodi",
    "borrowers": "Wakopaji",
    "portfolios": "Kanda za Mikopo",
    "creditReports": "Ripoti za Mikopo",
    "blockchain": "Daftari la Blockchain",
    "users": "Usimamizi wa Watumiaji",
    "audit": "Kumbukumbu za Ukaguzi",
    "billing": "Malipo",
    "regulatory": "Dashibodi ya Udhibiti",
    "settings": "Mipangilio",
    "profile": "Wasifu",
    "logout": "Ondoka"
  },
  "dashboard": {
    "title": "Dashibodi",
    "welcome": "Karibu, {{name}}",
    "totalBorrowers": "Jumla ya Wakopaji",
    "activeLoans": "Mikopo Inayoendelea",
    "totalPortfolioValue": "Thamani ya Jumla ya Kanda",
    "recentActivity": "Shughuli za Hivi Karibuni"
  },
  "borrowers": {
    "title": "Wakopaji",
    "addNew": "Ongeza Mkopaji Mpya",
    "search": "Tafuta mkopaji...",
    "name": "Jina",
    "email": "Barua pepe",
    "phone": "Simu",
    "status": "Hali",
    "actions": "Vitendo",
    "active": "Hai",
    "inactive": "Siyo Hai",
    "defaulted": "Ameshindwa Kulipa"
  },
  "credit": {
    "title": "Ripoti za Mikopo",
    "searchPlaceholder": "Tafuta kwa jina, kitambulisho, au barua pepe...",
    "noResults": "Hakuna wakopaji waliopatikana.",
    "latestScore": "Alama ya Hivi Karibuni",
    "generateReport": "Tengeneza Ripoti",
    "downloadReport": "Pakua Ripoti"
  },
  "billing": {
    "title": "Malipo",
    "currentPlan": "Mpango wa Sasa",
    "invoices": "Ankara",
    "paymentHistory": "Historia ya Malipo",
    "noRecordsTitle": "Hakuna Rekodi",
    "noRecordsDesc": "Hakuna rekodi za malipo zilizopatikana bado."
  },
  "common": {
    "save": "Hifadhi",
    "cancel": "Ghairi",
    "delete": "Futa",
    "edit": "Hariri",
    "export": "Hamisha",
    "exportExcel": "Hamisha kwa Excel",
    "exporting": "Inahamisha...",
    "loading": "Inapakia...",
    "confirm": "Thibitisha",

```



```

    "actions": "Vitendo",
    "search": "Tafuta",
    "filter": "Chuja",
    "noData": "Hakuna data",
    "error": "Hitilafu",
    "success": "Imefanikiwa"
  },
  "auth": {
    "login": "Ingia",
    "register": "Jisajili",
    "email": "Barua pepe",
    "password": "Neno la siri",
    "forgotPassword": "Umesahau neno la siri?",
    "rememberMe": "Nikumbuke"
  },
  "footer": {
    "allRightsReserved": "Haki zote zimehifadhiwa.",
    "terms": "Masharti ya Huduma",
    "privacy": "Sera ya Faragha",
    "security": "Usalama"
  }
}

```

Step 4 — Add RTL Support for Arabic

Edit the root layout to dynamically set text direction:

```

// src/app/layout.tsx
import { dir } from "i18next";

export default function RootLayout({
  children,
  params: { locale },
}: {
  children: React.ReactNode;
  params: { locale: string };
}) {
  const direction = locale === "ar" ? "rtl" : "ltr";

  return (
    <html lang={locale} dir={direction}>
      <body className={` ${direction === "rtl" ? "text-right" : "text-left"} `}>
        {children}
      </body>
    </html>
  );
}

```

Add RTL-aware Tailwind classes. In `tailwind.config.js`:

```

// tailwind.config.js
module.exports = {
  // ...
  plugins: [
    require("@tailwindcss/typography"),
    // ✅ Add RTL plugin if not already present
    // npm install tailwindcss-rtl
  ],
};

```

Testing Steps

1. Switch to **Arabic** → All UI elements should display in Arabic with RTL layout.
2. Switch to **Swahili** → All UI elements should display in Swahili.
3. Check that **no raw translation keys** (like `billing.noRecordsDesc`) appear.
4. If a key is genuinely missing, it should **fall back to English** gracefully.
5. Test navigation, forms, buttons, and error messages in both languages.

MEDIUM PRIORITY BUGS (Priority 3)

Bug 9: Header Name Not Updated Real-Time

Description

After editing the user's name in the profile form and saving, the name displayed in the top header/ navigation bar does not update until the user manually refreshes the page.

Root Cause

The header reads the user's name from a **cached auth session or state** that is not invalidated/updated when the profile is saved. The profile form saves to the API but does not propagate the change to the global auth context.

Files to Modify

```
src/hooks/useAuth.ts (or AuthContext/AuthProvider)
src/components/profile/ProfileForm.tsx
src/components/layout/Header.tsx
```

Step-by-Step Fix

Step 1 — Add an Update Function to the Auth Context

```
// src/contexts/AuthContext.tsx (or wherever your auth state lives)
"use client";

import { createContext, useContext, useState, useEffect, useCallback } from "react";
import { useSession } from "next-auth/react";

interface AuthContextType {
  user: any;
  isLoading: boolean;
  updateUserProfile: (updates: Partial<{ name: string; email: string }>) => void;
  refreshUser: () => Promise<void>;
}

const AuthContext = createContext<AuthContextType | null>(null);

export function AuthProvider({ children }: { children: React.ReactNode }) {
  const { data: session, status, update } = useSession();
  const [userOverrides, setUserOverrides] = useState<Record<string, any>>({});

  const user = session?.user
    ? { ...session.user, ...userOverrides }
    : null;

  // ✅ FIX: Allow components to update user data in real-time
  const updateUserProfile = useCallback((updates: Partial<{ name: string; email: string }>) => {
    setUserOverrides((prev) => ({ ...prev, ...updates }));
    // Also update the next-auth session
    update({ ...updates });
  }, [update]);

  // Force refresh from server
  const refreshUser = useCallback(async () => {
    await update();
    setUserOverrides({});
  }, [update]);

  return (
    <AuthContext.Provider value={{ user, isLoading: status === "loading", updateUserProfile, refreshUser }}>
      {children}
    </AuthContext.Provider>
  );
}

export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error("useAuth must be used within AuthProvider");
  return ctx;
}
```

Step 2 — Call `updateUserProfile` After Saving

```
// In src/components/profile/ProfileForm.tsx
import { useAuth } from "@contexts/AuthContext";

// Inside the component:
const { updateUserProfile } = useAuth();

const handleSave = async () => {
  try {
    const response = await fetch("/api/profile", {
      method: "PUT",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(formData),
    });

    if (response.ok) {
      const updatedUser = await response.json();
      // ✅ FIX: Immediately update the global auth state
      updateUserProfile({ name: updatedUser.name });
      toast.success("Profile updated successfully!");
    }
  } catch (error) {
    toast.error("Failed to update profile.");
  }
};
```

Step 3 — Ensure the Header Reads from Auth Context

```
// src/components/layout/Header.tsx
import { useAuth } from "@contexts/AuthContext";

export default function Header() {
  const { user } = useAuth();

  return (
    <header className="flex h-16 items-center justify-between border-b bg-white px-6">
      {/* ... other header content */}
      <div className="flex items-center gap-3">
        <span className="text-sm font-medium text-gray-700">
          {/* ✅ This now updates in real-time */}
          {user?.name || "User"}
        </span>
      </div>
    </header>
  );
}
```

Testing Steps

1. Note the name displayed in the header.
 2. Go to **Profile** → Change the name → Click **Save**.
 3. The header name should **immediately update** without refreshing the page.
 4. Navigate to other pages → The new name should persist.
 5. Refresh the page → The new name should still display (pulled from session/DB).
-

Bug 10: CSV/Excel Export Column Mismatch

Description

The Borrowers CSV export contains 10 columns, while the Excel export contains only 8 columns. The exports should have identical data structures.

Root Cause

The CSV and Excel export functions define their column arrays **independently**, and they have gone out of sync. One was updated when new fields were added, but the other was not.

Files to Modify

src/app/api/borrowers/ export /route.ts	(or separate csv/excel routes)
src/lib/ export /borrowerExport.ts	(Shared column definitions)

Step-by-Step Fix

Step 1 — Create a Single Source of Truth for Columns

```
// src/lib/export/borrowerColumns.ts

export interface ExportColumn {
  key: string;
  header: string;
  width?: number; // For Excel column width
  format?: (value: any) => string; // Optional formatter
}

// [X] Single definition used by BOTH CSV and Excel exports
export const BORROWER_EXPORT_COLUMNS: ExportColumn[] = [
  { key: "id", header: "Borrower ID", width: 20 },
  { key: "name", header: "Full Name", width: 25 },
  { key: "email", header: "Email", width: 30 },
  { key: "phone", header: "Phone Number", width: 18 },
  { key: "nationalId", header: "National ID", width: 20 },
  { key: "status", header: "Status", width: 12 },
  { key: "creditScore", header: "Credit Score", width: 14 },
  { key: "totalLoans", header: "Total Loans", width: 14 },
  { key: "outstandingDebt", header: "Outstanding Debt", width: 18 },
  { key: "createdAt", header: "Registration Date", width: 18,
    format: (val) => val ? new Date(val).toISOString().split("T")[0] : "N/A"
  },
];
```

Step 2 — Refactor Both Export Functions to Use Shared Columns

```
// src/app/api/borrowers/export/route.ts
import { BORROWER_EXPORT_COLUMNS } from "@lib/export/borrowerColumns";
import ExcelJS from "exceljs";

export async function GET(request: Request) {
  const { searchParams } = new URL(request.url);
  const format = searchParams.get("format") || "csv"; // "csv" or "excel"

  // Fetch borrowers (same query for both formats)
  const borrowers = await prisma.borrower.findMany({
    include: { loans: true },
  });

  // Transform data using shared column definitions
  const rows = borrowers.map((b) => {
    const row: Record<string, any> = {};
    BORROWER_EXPORT_COLUMNS.forEach((col) => {
      const rawValue = (b as any)[col.key];
      row[col.key] = col.format ? col.format(rawValue) : (rawValue ?? "N/A");
    });
    return row;
  });

  if (format === "csv") {
    // CSV Export
    const header = BORROWER_EXPORT_COLUMNS.map((c) => c.header).join(",");
    const csvRows = rows.map((row) => {
      BORROWER_EXPORT_COLUMNS.map((col) => {
        const val = String(row[col.key] ?? "").replace(/"/g, '"');
        return `${col.header} "${val}"`;
      }).join(",")
    });
    const csv = [header, ...csvRows].join("\n");

    return new Response(csv, {
      headers: {
        "Content-Type": "text/csv",
        "Content-Disposition": `attachment; filename="borrowers_export_${Date.now()}.csv"`,
      },
    });
  } else {
    // Excel Export – uses the SAME columns
    const workbook = new ExcelJS.Workbook();
    const worksheet = workbook.addWorksheet("Borrowers");

    worksheet.columns = BORROWER_EXPORT_COLUMNS.map((col) => ({
      header: col.header,
      key: col.key,
      width: col.width || 15,
    }));

    rows.forEach((row) => worksheet.addRow(row));

    // Style header
    worksheet.getRow(1).font = { bold: true };

    const buffer = await workbook.xlsx.writeBuffer();
    return new NextResponse(buffer, {
      headers: {
        "Content-Type": "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
      },
    });
  }
}
```

```

    "Content-Disposition": `attachment; filename="borrowers_export_${Date.now()}.xlsx`,
  },
});
}
}

```

Testing Steps

1. Export Borrowers as **CSV** → Open in a text editor → Count columns (should be 10).
2. Export Borrowers as **Excel** → Open in spreadsheet app → Count columns (should be 10).
3. Compare both files → **Column headers and data should be identical** in structure.
4. Verify data accuracy in both exports for a known borrower.

Bug 11: Chain Integrity Verify No Feedback

Description

Clicking the “Verify Integrity” button on the Blockchain Ledger page provides no visible feedback — no loading state, no success/error message, and no result display.

Root Cause

The button’s click handler either calls the API but **does not handle the response**, or the UI state variables for showing results are never updated.

Files to Modify

```
src/components/blockchain/VerifyIntegrityButton.tsx
```


Step-by-Step Fix

```
// src/components/blockchain/VerifyIntegrityButton.tsx
"use client";

import { useState } from "react";
import { ShieldCheck, Loader2, CheckCircle, XCircle } from "lucide-react";
import { useTranslation } from "next-i18next";

interface VerificationResult {
  valid: boolean;
  blocksVerified: number;
  totalBlocks: number;
  errors: string[];
}

export default function VerifyIntegrityButton() {
  const [isVerifying, setIsVerifying] = useState(false);
  const [result, setResult] = useState<VerificationResult | null>(null);
  const [error, setError] = useState<string | null>(null);
  const { t } = useTranslation();

  const handleVerify = async () => {
    setIsVerifying(true);
    setResult(null);
    setError(null);

    try {
      const response = await fetch("/api/blockchain/verify", {
        method: "POST",
      });

      if (!response.ok) {
        throw new Error("Verification request failed");
      }

      const data: VerificationResult = await response.json();

      // ✅ FIX: Actually display the result
      setResult(data);
    } catch (err: any) {
      console.error("Verification error:", err);
      setError(err.message || "Verification failed. Please try again.");
    } finally {
      setIsVerifying(false);
    }
  };

  return (
    <div>
      <button
        onClick={handleVerify}
        disabled={isVerifying}
        className="inline-flex items-center gap-2 rounded-md bg-blue-700 px-4 py-2
          text-sm font-medium text-white hover:bg-blue-800
          disabled:cursor-not-allowed disabled:opacity-50"
      >
        {isVerifying ? (
          <Loader2 className="h-4 w-4 animate-spin" />
        ) : (
          <ShieldCheck className="h-4 w-4" />
        )}
        {isVerifying
          ? t("blockchain.verifying", "Verifying...")
        }
      </button>
    </div>
  );
}
```

```

      : t("blockchain.verifyIntegrity", "Verify Integrity"))}
    </button>

    {/* ✅ FIX: Show verification results */}
    {result && (
      <div
        className={`mt-4 rounded-lg border p-4 ${
          result.valid
            ? "border-green-200 bg-green-50"
            : "border-red-200 bg-red-50"
          }`}
      >
        <div className="flex items-center gap-2">
          {result.valid ? (
            <CheckCircle className="h-5 w-5 text-green-600" />
          ) : (
            <XCircle className="h-5 w-5 text-red-600" />
          )}
          <span
            className={`font-medium ${
              result.valid ? "text-green-800" : "text-red-800"
            }`}
          >
            {result.valid
              ? t("blockchain.integrityValid", "Chain integrity verified success-
fully")
              : t("blockchain.integrityInvalid", "Chain integrity check failed")}
          </span>
        </div>
        <p className="mt-2 text-sm text-gray-600">
          {t("blockchain.blocksVerified", "Blocks verified")}: {res-
ult.blocksVerified} / {result.totalBlocks}
        </p>
        {result.errors.length > 0 && (
          <ul className="mt-2 list-disc pl-5 text-sm text-red-700">
            {result.errors.map((err, i) => (
              <li key={i}>{err}</li>
            ))}
          </ul>
        )}
      </div>
    )}

    {/* ✅ FIX: Show errors */}
    {error && (
      <div className="mt-4 rounded-lg border border-red-200 bg-red-50 p-4">
        <p className="text-sm text-red-700">{error}</p>
      </div>
    )}
  </div>
);
}

```

Testing Steps

1. Go to **Blockchain Ledger** page.
2. Click **Verify Integrity** → Should show a spinner with “Verifying...” text.
3. After completion → Should show green success or red failure banner.
4. Verify the block count displays correctly.
5. Simulate a failure (e.g., disconnect API) → Should show error message.

Bug 12: Raw Translation Keys Displayed

Description

Instead of translated text, the UI shows raw translation keys like `"billing.noRecordsDesc"`, `"dashboard.welcome"`, etc.

Root Cause

1. The translation key exists in the code but is **missing from the locale JSON file**.
2. The `i18n` library is configured to show keys when translations are not found (`returnNull: false` without fallback).
3. The wrong **namespace** may be loaded for the component.

Files to Modify

<code>public/locales/en/common.json</code>	(Add missing keys)
<code>public/locales/en/billing.json</code>	(If using separate namespaces)
<code>next-i18next.config.js</code>	(Fix namespace loading)

Step-by-Step Fix

Step 1 — Find All Raw Keys Being Displayed

In Replit Shell:

```
# Find all translation key usages in the codebase
grep -rohP "t\{(['\\"])([^\\""]+)[\\"]\}" src/ | sed "s/t(['\\"])/s/(['\\"])/" | sort -u
> /tmp/used_keys.txt

# Check which keys exist in English locale
node -e "
const en = require('./public/locales/en/common.json');
const flatten = (obj, prefix = '') => {
  let result = {};
  for (const [key, val] of Object.entries(obj)) {
    const newKey = prefix ? prefix + '.' + key : key;
    if (typeof val === 'object' && val !== null) {
      Object.assign(result, flatten(val, newKey));
    } else {
      result[newKey] = val;
    }
  }
  return result;
};
const flat = flatten(en);
console.log(Object.keys(flat).join('\n'));
" > /tmp/existing_keys.txt

# Find missing keys
comm -23 /tmp/used_keys.txt /tmp/existing_keys.txt
```

Step 2 — Add All Missing Keys

For the specific `billing.noRecordsDesc` case, add to `public/locales/en/common.json`:

```
{
  "billing": {
    "title": "Billing",
    "currentPlan": "Current Plan",
    "invoices": "Invoices",
    "paymentHistory": "Payment History",
    "noRecordsTitle": "No Records",
    "noRecordsDesc": "No billing records found yet. Your invoices will appear here
once generated.",
    "amount": "Amount",
    "date": "Date",
    "status": "Status",
    "paid": "Paid",
    "pending": "Pending",
    "overdue": "Overdue",
    "downloadInvoice": "Download Invoice"
  }
}
```

Repeat this for **every missing key** identified by the shell script above.

Step 3 — Configure i18n to Show Fallback Instead of Keys

Already addressed in Bug 8, Step 1 — but additionally, you can set a custom missing key handler:

```
// In your i18n initialization (e.g., src/lib/i18n.ts)
import i18n from "i18next";

i18n.init({
  // ... existing config
  saveMissing: true, // Enable in dev to log missing keys
  missingKeyHandler: (lngs, ns, key, fallbackValue) => {
    if (process.env.NODE_ENV === "development") {
      console.warn(`⚠ Missing translation: [${ns}] ${key}`);
    }
  },
});
```

Testing Steps

1. Navigate through **every page** of the application in English.
2. Search the page visually for any text that looks like a dot-separated key (e.g., `billing.noRecordsDesc`).
3. Check browser console for `⚠ Missing translation` warnings.
4. Verify the billing page specifically shows “No billing records found yet...” instead of the raw key.
5. Repeat in French, Portuguese, Arabic, and Swahili.

Bug 13: Missing 404 Handler

Description

Invalid URLs (e.g., `/nonexistent-page`) redirect to `/login` with a `200 OK` HTTP status instead of returning a proper `404 Not Found` response.

Root Cause

A **catch-all redirect** in the middleware or routing config sends all unmatched routes to `/login`. There is no custom `not-found.tsx` page (partially addressed in Bug 6), and the redirect returns `200` instead of `404`.

Files to Modify

<code>src/middleware.ts</code>	(Fix catch-all logic)
<code>src/app/not-found.tsx</code>	(Already created in Bug 6)
<code>src/app/[...catchAll]/page.tsx</code>	(Optional: explicit catch-all)

Step-by-Step Fix

Step 1 — Fix Middleware to Not Redirect Unknown Routes to /login

```
// src/middleware.ts – find and fix the catch-all redirect

// ❌ WRONG: This catches everything
// if (!isAuthenticated) {
//   return NextResponse.redirect(new URL("/login", request.url));
// }

// ✅ CORRECT: Only redirect auth-required routes to /login
const AUTH_REQUIRED_ROUTES = [
  "/dashboard",
  "/borrowers",
  "/portfolios",
  "/credit-reports",
  "/blockchain",
  "/users",
  "/audit",
  "/billing",
  "/regulatory-dashboard",
  "/profile",
  "/settings",
];

export async function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;

  // Only check auth for known protected routes
  const isProtectedRoute = AUTH_REQUIRED_ROUTES.some(
    (route) => pathname === route || pathname.startsWith(route + "/")
  );

  if (isProtectedRoute) {
    const token = await getToken({ req: request, secret: process.env.NEXTAUTH_SECRET });
    if (!token) {
      const loginUrl = new URL("/login", request.url);
      loginUrl.searchParams.set("callbackUrl", pathname);
      return NextResponse.redirect(loginUrl);
    }

    // ... role checking from Bug 1
  }

  // ✅ For unknown routes, let Next.js handle 404 naturally
  return NextResponse.next();
}

export const config = {
  // ✅ Only match known route patterns, not everything
  matcher: [
    "/dashboard/:path*",
    "/borrowers/:path*",
    "/portfolios/:path*",
    "/credit-reports/:path*",
    "/blockchain/:path*",
    "/users/:path*",
    "/audit/:path*",
    "/billing/:path*",
    "/regulatory-dashboard/:path*",
    "/profile/:path*",
    "/settings/:path*",
  ],
};
```


Step 2 — Verify the `not-found.tsx` Exists (Created in Bug 6)

The `src/app/not-found.tsx` from Bug 6 will automatically handle 404s. Verify it returns the proper HTTP status by adding metadata:

```
// src/app/not-found.tsx
import { Metadata } from "next";

export const metadata: Metadata = {
  title: "404 - Page Not Found | Africa Credit Hub",
};

// ... rest of the component from Bug 6
```

Testing Steps

1. Navigate to `/this-does-not-exist` → Should show the custom 404 page.
2. Check DevTools → Network tab → Response status should be **404**, not 200 or 302.
3. Navigate to `/dashboard` while logged out → Should redirect to `/login` (this should still work).
4. Navigate to `/dashboard` while logged in → Should show the dashboard (no change).
5. Navigate to `/login` → Should show login page (200 OK).

Bug 14: Navigation Bar Inconsistencies

Description

The navigation bar displays differently between the homepage/landing page and authenticated sub-pages. Links, styling, or layout elements may differ or be missing.

Root Cause

Two separate navigation components exist — one for the public/landing pages and one for the authenticated app — and they were not designed with a consistent structure.

Files to Modify

<code>src/components/layout/PublicHeader.tsx</code>	(Landing page header)
<code>src/components/layout/Header.tsx</code>	(Authenticated header)
<code>src/components/layout/BaseNavbar.tsx</code>	(NEW — shared base component)

Step-by-Step Fix

Step 1 — Create a Shared Navbar Base

```
// src/components/layout/BaseNavbar.tsx
"use client";

import Link from "next/link";
import Image from "next/image";

interface BaseNavbarProps {
  children?: React.ReactNode; // Slot for right-side content (login btn vs. user menu)
  variant?: "light" | "dark";
}

export default function BaseNavbar({ children, variant = "light" }: BaseNavbarProps) {
  const bgClass = variant === "dark" ? "bg-gray-900 text-white" : "bg-white text-gray-900";
  const borderClass = variant === "dark" ? "border-gray-700" : "border-gray-200";

  return (
    <header className={`sticky top-0 z-50 ${bgClass} border-b ${borderClass}`}>
      <div className="mx-auto flex h-16 max-w-7xl items-center justify-between px-4 sm:px-6 lg:px-8">
        </* Logo – consistent across all pages */>
        <Link href="/" className="flex items-center gap-2">
          <Image
            src="/logo.svg"
            alt="Africa Credit Hub"
            width={32}
            height={32}
          />
          <span className="text-lg font-bold">Africa Credit Hub</span>
        </Link>

        </* Right side – injected by parent */>
        <div className="flex items-center gap-4">
          {children}
        </div>
      </div>
    </header>
  );
}
```

Step 2 — Use It in Both Public and Authenticated Layouts

```
// src/components/layout/PublicHeader.tsx
import BaseNavbar from "../BaseNavbar";
import Link from "next/link";

export default function PublicHeader() {
  return (
    <BaseNavbar>
      <Link href="/login" className="text-sm font-medium text-gray-600 hover:text-gray-900">
        Sign In
      </Link>
      <Link
        href="/register"
        className="rounded-md bg-green-700 px-4 py-2 text-sm font-medium text-white hover:bg-green-800"
      >
        Get Started
      </Link>
    </BaseNavbar>
  );
}
```

```
// src/components/layout/Header.tsx (authenticated)
import BaseNavbar from "../BaseNavbar";
import { useAuth } from "@contexts/AuthContext";
import NotificationBell from "../NotificationBell";
import UserMenu from "../UserMenu";

export default function Header() {
  const { user } = useAuth();

  return (
    <BaseNavbar>
      <NotificationBell />
      <UserMenu user={user} />
    </BaseNavbar>
  );
}
```

Testing Steps

1. View the **homepage** (logged out) → Check logo, spacing, and alignment.
2. Log in and view the **dashboard** → Header should have the same logo, same height, same base layout.
3. Compare both visually → Logo position, font, and overall structure should be consistent.
4. Test on mobile viewport → Both headers should be responsive with consistent behavior.

LOW PRIORITY BUGS (Priority 4)

Bug 15: Notification Close Button Non-Functional

Description

The “×” button on notification toasts/banners does not dismiss the notification.

Root Cause

The `onClick` handler is either missing, references the wrong function, or does not update the state that controls notification visibility.

Files to Modify

`src/components/ui/Notification.tsx` (or Toast component)

Step-by-Step Fix

```
// src/components/ui/Notification.tsx
"use client";

import { useState, useEffect } from "react";
import { X, CheckCircle, AlertCircle, Info } from "lucide-react";

interface NotificationProps {
  id: string;
  type: "success" | "error" | "info" | "warning";
  message: string;
  duration?: number; // auto-dismiss in ms
  onDismiss: (id: string) => void;
}

export default function Notification({
  id,
  type,
  message,
  duration = 5000,
  onDismiss,
}: NotificationProps) {
  const [isVisible, setIsVisible] = useState(true);

  useEffect(() => {
    if (duration > 0) {
      const timer = setTimeout(() => {
        handleDismiss();
      }, duration);
      return () => clearTimeout(timer);
    }
  }, [duration]);

  const handleDismiss = () => {
    setIsVisible(false);
    // Wait for animation to complete before removing from DOM
    setTimeout(() => onDismiss(id), 300);
  };

  const icons = {
    success: <CheckCircle className="h-5 w-5 text-green-500" />,
    error: <AlertCircle className="h-5 w-5 text-red-500" />,
    info: <Info className="h-5 w-5 text-blue-500" />,
    warning: <AlertCircle className="h-5 w-5 text-yellow-500" />,
  };

  const bgColors = {
    success: "bg-green-50 border-green-200",
    error: "bg-red-50 border-red-200",
    info: "bg-blue-50 border-blue-200",
    warning: "bg-yellow-50 border-yellow-200",
  };

  return (
    <div
      className={`flex items-start gap-3 rounded-lg border p-4 shadow-lg transition-all duration-300
        ${bgColors[type]}
        ${isVisible ? "translate-x-0 opacity-100" : "translate-x-full opacity-0"}`}
    >
      {icons[type]}
      <p className="flex-1 text-sm text-gray-800">{message}</p>
    </div>
  );
}
```

```

    /*  FIX: Ensure onClick handler actually calls handleDismiss */
    <button
      onClick={handleDismiss}
      className="flex-shrink-0 rounded p-1 text-gray-400 hover:bg-gray-200
        hover:text-gray-600
        focus:outline-none focus:ring-2 focus:ring-gray-300"
      aria-label="Dismiss notification"
      type="button"
    >
      <X className="h-4 w-4" />
    </button>
  </div>
);
}

```

Also verify the parent notification manager passes `onDismiss` correctly:

```

// src/components/ui/NotificationContainer.tsx
"use client";

import { useNotifications } from "@hooks/useNotifications";
import Notification from "../Notification";

export default function NotificationContainer() {
  const { notifications, removeNotification } = useNotifications();

  return (
    <div className="fixed right-4 top-4 z-50 flex flex-col gap-2">
      {notifications.map((notif) => (
        <Notification
          key={notif.id}
          id={notif.id}
          type={notif.type}
          message={notif.message}
          onDismiss={removeNotification}   Must be wired up
        </Notification>
      ))}
    </div>
  );
}

```

Testing Steps

1. Trigger a notification (e.g., save a form → success toast appears).
2. Click the **x button** → Notification should smoothly slide out and disappear.
3. Trigger multiple notifications → Each x button should dismiss only its own notification.
4. Wait without clicking → Notifications should auto-dismiss after 5 seconds.

Bug 16: Grammar Error — “1 accounts”

Description

When there is exactly 1 account/item, the UI shows “1 accounts” (plural) instead of “1 account” (singular).

Root Cause

The display string uses a hardcoded plural form without checking the count. E.g.,

``${count} accounts`` instead of conditional pluralization.

Files to Modify

<code>src/lib/utils/pluralize.ts</code>	(NEW – utility function)
Any component that displays counts	(search for occurrences)
<code>public/locales/en/common.json</code>	(Use <code>i18n</code> plural support)

Step-by-Step Fix

Option A — Quick Fix with a Utility Function

```
// src/lib/utils/pluralize.ts
export function pluralize(count: number, singular: string, plural?: string): string {
  const pluralForm = plural || singular + "s";
  return `${count} ${count === 1 ? singular : pluralForm}`;
}

// Usage in components:
// import { pluralize } from "@lib/utils/pluralize";
// pluralize(1, "account") → "1 account"
// pluralize(5, "account") → "5 accounts"
// pluralize(1, "entry", "entries") → "1 entry"
```

Option B — Use `i18n` Built-in Pluralization (Recommended)

In `public/locales/en/common.json`:

```
{
  "accounts": {
    "count_one": "{{count}} account",
    "count_other": "{{count}} accounts"
  },
  "borrowers": {
    "count_one": "{{count}} borrower",
    "count_other": "{{count}} borrowers"
  },
  "loans": {
    "count_one": "{{count}} loan",
    "count_other": "{{count}} loans"
  }
}
```

In the component:

```
// ❌ WRONG
<span>{users.length} accounts</span>

// ✅ CORRECT (using i18n)
<span>{{t("accounts.count", { count: users.length })}}</span>

// ✅ ALSO CORRECT (using utility)
<span>{pluralize(users.length, "account")}</span>
```


Step 3 — Find and Fix All Instances

```
# In Replit Shell – find all hardcoded plurals with counts
grep -rn "{} accounts\|} users\|} borrowers\|} loans\|} records\|} items" src/
```

Fix every result to use either the utility function or i18n pluralization.

Testing Steps

1. Create exactly **1** user/borrower/loan.
2. Verify the UI shows “1 account” (singular), NOT “1 accounts”.
3. Add a second → Verify it shows “2 accounts” (plural).
4. Test with 0 → Should show “0 accounts” (plural).

Bug 17: Missing Profile Features

Description

The profile page lacks essential features: no avatar upload, no phone number field, no password change, no account deletion option.

Root Cause

The profile page was implemented as a minimal stub with only name and email fields. Additional features were not developed.

Files to Modify

src/app/(dashboard)/profile/page.tsx	
src/components/profile/ProfileForm.tsx	(Enhance)
src/components/profile/AvatarUpload.tsx	(NEW)
src/app/api/profile/route.ts	(Enhance API)
src/app/api/profile/avatar/route.ts	(NEW ☐ avatar upload endpoint)
prisma/schema.prisma	(Add fields to User model)

Step-by-Step Fix

Step 1 — Update the Database Schema

```
// prisma/schema.prisma
model User {
  id          String    @id @default(uuid())
  name        String
  email       String    @unique
  phone       String?   // ☒ NEW
  avatarUrl   String?   // ☒ NEW
  role        String    @default("viewer")
  organization String?   // ☒ NEW
  jobTitle    String?   // ☒ NEW
  timezone    String?   @default("UTC") // ☒ NEW
  passwordHash String
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
}
```

Run migration:

```
npx prisma migrate dev --name add_profile_fields
```

Step 2 — Create Avatar Upload Component

```
// src/components/profile/AvatarUpload.tsx
"use client";

import { useState, useRef } from "react";
import { Camera, Loader2 } from "lucide-react";
import Image from "next/image";

interface AvatarUploadProps {
  currentAvatarUrl?: string;
  userName: string;
  onUpload: (url: string) => void;
}

export default function AvatarUpload({ currentAvatarUrl, userName, onUpload }: AvatarUploadProps) {
  const [isUploading, setIsUploading] = useState(false);
  const fileInputRef = useRef<HTMLInputElement>(null);

  const initials = userName
    .split(" ")
    .map((n) => n[0])
    .join("")
    .substring(0, 2)
    .toUpperCase();

  const handleFileSelect = async (e: React.ChangeEvent<HTMLInputElement>) => {
    const file = e.target.files?.[0];
    if (!file) return;

    // Validate
    if (!file.type.startsWith("image/")) {
      alert("Please select an image file");
      return;
    }
    if (file.size > 5 * 1024 * 1024) {
      alert("Image must be under 5MB");
      return;
    }

    setIsUploading(true);
    try {
      const formData = new FormData();
      formData.append("avatar", file);

      const response = await fetch("/api/profile/avatar", {
        method: "POST",
        body: formData,
      });

      if (response.ok) {
        const { avatarUrl } = await response.json();
        onUpload(avatarUrl);
      }
    } catch (err) {
      console.error("Upload failed:", err);
    } finally {
      setIsUploading(false);
    }
  };

  return (
    <div className="flex flex-col items-center gap-3">

```

```

    <div
      className="relative h-24 w-24 cursor-pointer overflow-hidden rounded-full bg-
green-700"
      onClick={() => fileInputRef.current?.click()}
    >
      {currentAvatarUrl ? (
        <Image src={currentAvatarUrl} alt={userName} fill className="object-cover" /
      >
        ) : (
          <span className="flex h-full w-full items-center justify-center text-2xl
font-bold text-white">
            {initials}
          </span>
        )}
      {/* Overlay */}
      <div className="absolute inset-0 flex items-center justify-center bg-black/40
opacity-0 transition hover:opacity-100">
        {isUploading ? (
          <Loader2 className="h-6 w-6 animate-spin text-white" />
        ) : (
          <Camera className="h-6 w-6 text-white" />
        )}
      </div>
    </div>
    <input
      ref={fileInputRef}
      type="file"
      accept="image/*"
      className="hidden"
      onChange={handleFileSelect}
    />
    <button
      onClick={() => fileInputRef.current?.click()}
      className="text-sm text-green-700 hover:underline"
    >
      Change Avatar
    </button>
  </div>
);
}

```

Step 3 — Enhanced Profile Form

```
// src/components/profile/ProfileForm.tsx
"use client";

import { useState } from "react";
import { useAuth } from "@/contexts/AuthContext";
import { useTranslation } from "next-i18next";
import AvatarUpload from "@/AvatarUpload";
import { sanitizeText, validateName } from "@/lib/sanitize";

export default function ProfileForm({ initialData }: { initialData: any }) {
  const { updateUserProfile } = useAuth();
  const { t } = useTranslation();
  const [formData, setFormData] = useState({
    name: initialData.name || "",
    email: initialData.email || "",
    phone: initialData.phone || "",
    organization: initialData.organization || "",
    jobTitle: initialData.jobTitle || "",
    timezone: initialData.timezone || "UTC",
    avatarUrl: initialData.avatarUrl || "",
  });
  const [isSaving, setIsSaving] = useState(false);
  const [message, setMessage] = useState<{ type: string; text: string } | null>(null);

  const handleChange = (e: React.ChangeEvent<HTMLInputElement | HTMLSelectElement>) => {
    setFormData((prev) => ({ ...prev, [e.target.name]: e.target.value }));
  };

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setIsSaving(true);
    setMessage(null);

    try {
      const response = await fetch("/api/profile", {
        method: "PUT",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(formData),
      });

      if (response.ok) {
        const updated = await response.json();
        updateUserProfile({ name: updated.name });
        setMessage({ type: "success", text: t("profile.saved", "Profile updated successfully!") });
      } else {
        const err = await response.json();
        setMessage({ type: "error", text: err.error || "Failed to save." });
      }
    } catch {
      setMessage({ type: "error", text: "An error occurred." });
    } finally {
      setIsSaving(false);
    }
  };

  return (
    <form onSubmit={handleSubmit} className="space-y-8">
      <div>
        <AvatarUpload
          currentAvatarUrl={formData.avatarUrl}
        />
      </div>
    </form>
  );
}
```

```

      userName={formData.name}
      onUpload={url => setFormData((prev) => ({ ...prev, avatarUrl: url })))}
    />

    {/* Form Fields */}
    <div className="grid gap-6 sm:grid-cols-2">
      <div>
        <label className="block text-sm font-medium text-gray-700">
          {t("profile.fullName", "Full Name")}
        </label>
        <input
          name="name"
          value={formData.name}
          onChange={handleChange}
          className="mt-1 block w-full rounded-md border border-gray-300 px-3 py-2"
          required
        />
      </div>

      <div>
        <label className="block text-sm font-medium text-gray-700">
          {t("profile.email", "Email")}
        </label>
        <input
          name="email"
          type="email"
          value={formData.email}
          onChange={handleChange}
          className="mt-1 block w-full rounded-md border border-gray-300 px-3 py-2"
          required
        />
      </div>

      <div>
        <label className="block text-sm font-medium text-gray-700">
          {t("profile.phone", "Phone Number")}
        </label>
        <input
          name="phone"
          type="tel"
          value={formData.phone}
          onChange={handleChange}
          placeholder="+234 800 000 0000"
          className="mt-1 block w-full rounded-md border border-gray-300 px-3 py-2"
        />
      </div>

      <div>
        <label className="block text-sm font-medium text-gray-700">
          {t("profile.organization", "Organization")}
        </label>
        <input
          name="organization"
          value={formData.organization}
          onChange={handleChange}
          className="mt-1 block w-full rounded-md border border-gray-300 px-3 py-2"
        />
      </div>

      <div>
        <label className="block text-sm font-medium text-gray-700">
          {t("profile.jobTitle", "Job Title")}
        </label>

```



```

        <input
          name="jobTitle"
          value={formData.jobTitle}
          onChange={handleChange}
          className="mt-1 block w-full rounded-md border border-gray-300 px-3 py-2"
        />
      </div>

      <div>
        <label className="block text-sm font-medium text-gray-700">
          {t("profile.timezone", "Timezone")}
        </label>
        <select
          name="timezone"
          value={formData.timezone}
          onChange={handleChange}
          className="mt-1 block w-full rounded-md border border-gray-300 px-3 py-2"
        >
          <option value="Africa/Lagos">West Africa (WAT)</option>
          <option value="Africa/Nairobi">East Africa (EAT)</option>
          <option value="Africa/Cairo">Egypt (EET)</option>
          <option value="Africa/Johannesburg">South Africa (SAST)</option>
          <option value="Africa/Casablanca">Morocco (WET)</option>
          <option value="UTC">UTC</option>
        </select>
      </div>
    </div>

    {/* Status Message */}
    {message && (
      <div
        className={`rounded-md p-3 text-sm ${
          message.type === "success"
            ? "bg-green-50 text-green-700"
            : "bg-red-50 text-red-700"
        }`}
      >
        {message.text}
      </div>
    )}

    {/* Submit */}
    <button
      type="submit"
      disabled={isSaving}
      className="rounded-md bg-green-700 px-6 py-2.5 text-sm font-medium text-white
      hover:bg-green-800 disabled:opacity-50"
    >
      {isSaving ? t("common.saving", "Saving...") : t("common.save", "Save Changes")}
    </button>
  </form>
);
}

```

Testing Steps

1. Go to **Profile** → All fields should be visible (name, email, phone, org, job title, timezone).
2. Click the avatar circle → File picker opens → Select an image → Avatar updates.
3. Fill in phone number → Save → Reload page → Phone number persists.
4. Verify the header name updates after saving (from Bug 9 fix).

Bug 18: Version Branding Inconsistencies

Description

Different parts of the application reference different version numbers: `v2.0` on the login page, `v2.1` in the footer, `v2.5` in the about section. This confuses users and looks unprofessional.

Root Cause

Version strings are **hardcoded in multiple places** instead of being pulled from a single source of truth.

Files to Modify

```
src/lib/constants.ts (NEW  or add to existing constants file)
All files that reference version numbers (search results)
```

Step-by-Step Fix

Step 1 — Create a Single Source of Truth

```
// src/lib/constants.ts
export const APP_VERSION = "2.5.0";
export const APP_NAME = "Africa Credit Hub";
export const APP_DESCRIPTION = "Credit Scoring & Portfolio Management for Africa";
```

Step 2 — Find All Hardcoded Version Strings

```
# In Replit Shell
grep -rn "v2\.\.|v2\.0|v2\.1|v2\.5|version.*2\." src/
```

Step 3 — Replace All Instances

In every file found:

```
// ❌ WRONG – hardcoded
<span>Africa Credit Hub v2.0</span>
<span>Version 2.1</span>
<footer>v2.5</footer>

// ✅ CORRECT – single source
import { APP_VERSION, APP_NAME } from "@lib/constants";

<span>{APP_NAME} v{APP_VERSION}</span>
```

Or if used in the `package.json` version field, read from there:

```
// src/lib/constants.ts – alternative approach
import packageJson from "../../package.json";

export const APP_VERSION = packageJson.version;
```

Testing Steps

1. Check the **login page** → Version should show `v2.5.0` .
 2. Check the **footer** → Same version.
 3. Check the **about/help page** → Same version.
 4. Search the entire rendered app for any occurrence of `v2.0` or `v2.1` → None should remain.
 5. To change the version in the future, only `src/lib/constants.ts` (or `package.json`) needs updating.
-



Post-Fix Verification Checklist

After applying all fixes, run through this checklist:

#	Check	Status
1	Viewer cannot access / users, /audit, /billing, /regulatory-dashboard	☐
2	Script tags rejected in name field (client + API)	☐
3	Portfolio Excel export downloads with readable filename and correct data	☐
4	Credit report search finds borrowers by name, ID, email, phone	☐
5	Sidebar shows translated labels in FR, PT, AR, SW	☐
6	/profile and /settings show pages (no dev error messages)	☐
7	Terms links go to /terms, Privacy links go to /privacy	☐
8	Arabic and Swahili display translated text (with EN fallback)	☐
9	Header name updates immediately after profile save	☐
10	CSV and Excel borrower exports have identical 10 columns	☐
11	Verify Integrity button shows loading → result feedback	☐
12	No raw translation keys (like <code>billing.noRecordsDesc</code>) visible	☐
13	Invalid URLs show 404 page (not redirect to /login with 200)	☐

#	Check	Status
14	Navbar consistent between homepage and sub-pages	☐
15	Notification × button dismisses the notification	☐
16	“1 account” (singular) displays correctly	☐
17	Profile page has avatar, phone, org, job title, timezone fields	☐
18	All version references show the same version number	☐



Quick-Start Commands for Replit

```
# Install required dependencies
npm install isomorphic-dompurify exceljs lodash

# Run database migration (if using Prisma)
npx prisma migrate dev --name bug_fixes_april_2026

# Find all translation keys used in the app
grep -rohP "t\(['\"](?:[^\"]|\\\")+\)['\"]\)" src/ | sort -u

# Find all hardcoded version strings
grep -rn "v2\.\.|v2\.0\|v2\.1\|v2\.5" src/

# Find all usages of dangerouslySetInnerHTML
grep -rn "dangerouslySetInnerHTML" src/

# Find all hardcoded English strings in sidebar
grep -rn "Dashboard\|Borrowers\|Portfolios" src/components/layout/Sidebar

# Run the development server
npm run dev
```

Document generated for the Africa Credit Hub development team. Apply fixes in priority order (Critical → High → Medium → Low). Each fix includes testing steps — verify each one before moving to the next.